

# AI Live Meeting Summarizer

## Speech Recognition Model Evaluation Report

### 1. Introduction

Automatic Speech Recognition (ASR) and Natural Language Processing (NLP) are core technologies in modern Artificial Intelligence systems. With the rapid increase in virtual meetings (Zoom, Teams, Google Meet), there is a growing demand for systems that can automatically transcribe and summarize conversations.

The **AI Live Meeting Summarizer** is designed as an end-to-end intelligent system that converts raw meeting audio into structured summaries. The system integrates multiple AI components such as speech recognition, speaker diarization, and transformer-based summarization.

### 2. Objectives

The key objectives of this project include:

- Develop a pipeline for speech-to-text conversion
- Compare ASR models using Word Error Rate (WER)
- Implement real-time transcription
- Perform speaker diarization (who spoke when)
- Generate structured meeting summaries
- Evaluate performance using WER, DER, ROUGE, BLEU

### 3. System Architecture

The system follows a multi-stage pipeline:

1. Audio Input (Dataset / Microphone)
2. Audio Preprocessing
3. Speech Recognition (Vosk / Whisper)
4. Speaker Diarization (Pyannote)
5. Text Cleaning & Processing
6. Summarization (Transformer Model)
7. Evaluation Metrics

### 4. Dataset

The dataset used is the **AMI Meeting Corpus**, a real-world dataset containing multi-speaker meeting recordings.

**Audio File:**

- ES2002a.Array1-01.wav

**Reference Files:**

- ES2002a.A.words.xml
- ES2002a.B.words.xml
- ES2002a.C.words.xml
- ES2002a.D.words.xml

**5. Audio Preprocessing**

Audio preprocessing is performed to standardize input for ASR models.

**Steps:**

- Convert audio to 16kHz
- Convert stereo to mono
- Normalize audio format
- Process full-length meeting audio

**Output:**

- Processed audio ready for transcription

**6. Reference Transcript Generation**

Since transcripts are in XML format, preprocessing is required.

**Steps:**

- Parse XML files
- Extract words and timestamps
- Merge speaker transcripts
- Generate final reference text

**Output:**

- reference\_trimmed.txt (or full transcript)

**7. Speech Recognition Models****7.1 Vosk**

- Lightweight offline model

- Faster but less accurate

## 7.2 Whisper

- Transformer-based model
- Better contextual understanding

## 8. Word Error Rate (WER)

WER evaluates transcription accuracy:

$WER = (\text{Substitutions} + \text{Insertions} + \text{Deletions}) / \text{Total Words}$

**Interpretation:**

- 0 → Perfect
- 0.5 → Poor accuracy

## 9. Vosk Evaluation (Full Audio)

- Reference words: 384
- Predicted words: 238
- WER: 0.7578

**Analysis:**

- Struggles with noise
- Poor handling of multiple speakers

```
(venv) PS E:\Live-Meeting-Summarizer-Application> python services/evaluate_wer.py

REFERENCE SAMPLE:
hi im david and im supposed to be an industrial designer i just got the project announcement about what the project is designing a remote control thats about i
t didnt get anything else did you get the same thing cool theres too much gear great so were designing a new remote control and oh i have to

VOSK SAMPLE:
hi im david and im supposed to be an industrial uk manager and im a marketing experts predict this great so designing your control after the quarter hes here s
o im doing good i dont yes it is a design you continue to do test the sphere original trendy and user friendly so thats kind of pretty why tw

Reference word count: 384
Vosk word count: 238

Vosk Word Error Rate: 0.7578125
(venv) PS E:\Live-Meeting-Summarizer-Application> python services/evaluate_wer_whisper.py
```

## 10. Whisper Evaluation (Full Audio)

- Reference words: 384

- Predicted words: 342
- WER: 0.7109

#### Analysis:

- Better than Vosk
- Handles context better

```
(venv) PS E:\Live-Meeting-Summarizer-Application> python services/evaluate_wer_whisper.py

REFERENCE SAMPLE:
hi im david and im supposed to be an industrial designer i just got the project announcement about what the project is designing a remote control thats about i
t didnt get anything else did you get the same thing cool theres too much gear great so were designing a new remote control and oh i have to

WHISPER SAMPLE:
hi im david and im supposed to be in the destroyer these days im andrew and im marketing experts im greg and im a kneejumped face great im sort of designing a
new era of control and oh i have to record his hair actually so thats david and jim craig and you all arrive dont you yes which is a design i

Reference word count: 384
Whisper word count: 342

Whisper Word Error Rate: 0.7109375
```

## 11. Model Comparison

| Model   | WER    | Performance |
|---------|--------|-------------|
| Vosk    | 0.7578 | Average     |
| Whisper | 0.7109 | Better      |

```
(venv) PS E:\Live-Meeting-Summarizer-Application> python services/compare_models_wer.py

Reference words: 384
Vosk words: 238
Whisper words: 342

VOSK WER: 0.7578125
WHISPER WER: 0.7109375

Better Model: Whisper
(venv) PS E:\Live-Meeting-Summarizer-Application> █
```

## 12. Real-Time Speech Recognition

#### Process:

- Record audio using microphone
- Perform live transcription

**Output:**

"hello everyone welcome to the meeting"

**WER:**

0.0

**Interpretation:**

- Perfect accuracy in clean audio
- Shows system works well in controlled conditions

```
(venv310) PS E:\Live-Meeting-Summarizer-Application> python services/realtime_speech_wer.py
🔊 Recording will start in 1 second...
✅ Recording finished
Saved: test_recording.wav

Loading Vosk model...

🎤 Transcribing...

🗣️ Recognized: hello everyone welcome to the meeting

📄 Final transcription:
hello everyone welcome to the meeting

📊 WER: 0.0
```

### 13. Speaker Diarization

**Model:**

- pyannote/speaker-diarization-3.1

**Output:**

- Time-based speaker segments

**Example:**

- 15.07s – 15.18s : SPEAKER\_00

```

• (venv310) PS E:\Live-Meeting-Summarizer-Application> python services/speaker_diarization.py
Loading diarization model...
Loading audio...
Running diarization...

Speaker Segments:

15.07s - 15.18s : SPEAKER_00
15.22s - 15.84s : SPEAKER_00
19.20s - 19.29s : SPEAKER_03
19.29s - 19.79s : SPEAKER_02
19.79s - 19.93s : SPEAKER_03
20.57s - 21.28s : SPEAKER_00
25.75s - 26.88s : SPEAKER_01
26.95s - 27.06s : SPEAKER_01

```

## 14. Diarization Error Rate (DER)

### Results:

- DER: **29.07%(error)** , Accuracy : **(82%)**
- False Alarm: 11.23%
- Missed Detection: 15.61%
- Confusion: 2.23%

### Interpretation:

- Moderate performance
- Main errors: missed speech

```

(venv310) PS E:\Live-Meeting-Summarizer-Application> python diarization/compute_der.py
E:\Live-Meeting-Summarizer-Application\venv310\lib\site-packages\pyannote\metrics\utils.py:200: UserWarning: 'uem' was approximated by
the union of 'reference' and 'hypothesis' extents.
  warnings.warn(
Diarization Error Rate (DER):
diarization error rate  total correct correct false alarm false alarm missed detection missed detection confusion confusion
item                    %                                     %                                     %                                     %                                     %
ES2002a                29.07 946.42 777.57 82.16      106.26      11.23      147.71      15.61  21.13  2.23
TOTAL                  29.07 946.42 777.57 82.16      106.26      11.23      147.71      15.61  21.13  2.23
(venv310) PS E:\Live-Meeting-Summarizer-Application>

```

## 15. Meeting Summarization

### Model:

- BART (Transformer)

### Output:

- Key points
- Action items
- Decisions

## Scores:

- ROUGE F1  $\approx 0.28$
- BLEU  $\approx 0.007$

## Interpretation:

- Captures meaning
- Needs improvement

## Meeting Summary:

David and Jim Craig are designing a new era of control. The project is a design in your control. There are three different stages to the design. The spear-riching old trendy and user-friendly is the brief. You can take as long as you because we haven't got an awful lot of physical affection. I could totally hold up more but beagles. And then the small kitten for a. And, um, my butt here, it's come real.

```
(venv310) PS E:\Live-Meeting-Summarizer-Application> python summarization/evaluate_summary.py
Starting evaluation...

Generated Summary:

Meeting Summary
-----

Key Discussion Points:
David and Jim Craig are designing a new era of control. The project is a design in your control. There are three different stages to the design. The spear-riching old tr
user-friendly is the brief. You can take as long as you because we haven't got an awful lot of physical affection. I could totally hold up more but beagles. And then th
kitten for a. And, um, my butt here, it's come real.

Action Items:
• Review the design stages discussed in the meeting
• Continue refining the design concepts

Decisions:
• Proceed with the proposed design framework

ROUGE Scores:
{'rouge1': Score(precision=0.19626168224299065, recall=0.5, fmeasure=0.28187919463087246), 'rougeL': Score(precision=0.12149532710280374, recall=0.30952380952380953, fme
174496644295302)}

BLEU Score:
0.00723838466670598
(venv310) PS E:\Live-Meeting-Summarizer-Application>
```

## 16. Reasons for High Error Rates

1. Multi-speaker environment
2. Background noise
3. Far-field microphones
4. Speaker overlap

## 17. Pipeline Integration

The AI Live Meeting Summarizer is designed as an end-to-end pipeline where multiple AI components work together to process meeting audio and generate structured summaries.

### Pipeline Flow

1. **Audio Input**
  - Pre-recorded audio (AMI dataset) or real-time microphone input
2. **Audio Preprocessing**

- Convert to 16kHz and mono
  - Ensures compatibility with ASR models
- 3. Speech Recognition (ASR)**
    - Vosk (real-time) and Whisper (accurate)
    - Output: Raw transcript
  - 4. Speaker Diarization**
    - Using Pyannote
    - Output: Speaker-labeled segments
  - 5. Transcript Alignment**
    - Combine transcription with speaker segments
    - Output: Speaker-aware transcript
  - 6. Text Processing & Summarization**
    - Clean and structure text
    - BART generates key points, actions, and decisions
  - 7. Evaluation Metrics**
    - WER → Transcription accuracy
    - DER → Speaker accuracy
    - ROUGE/BLEU → Summary quality

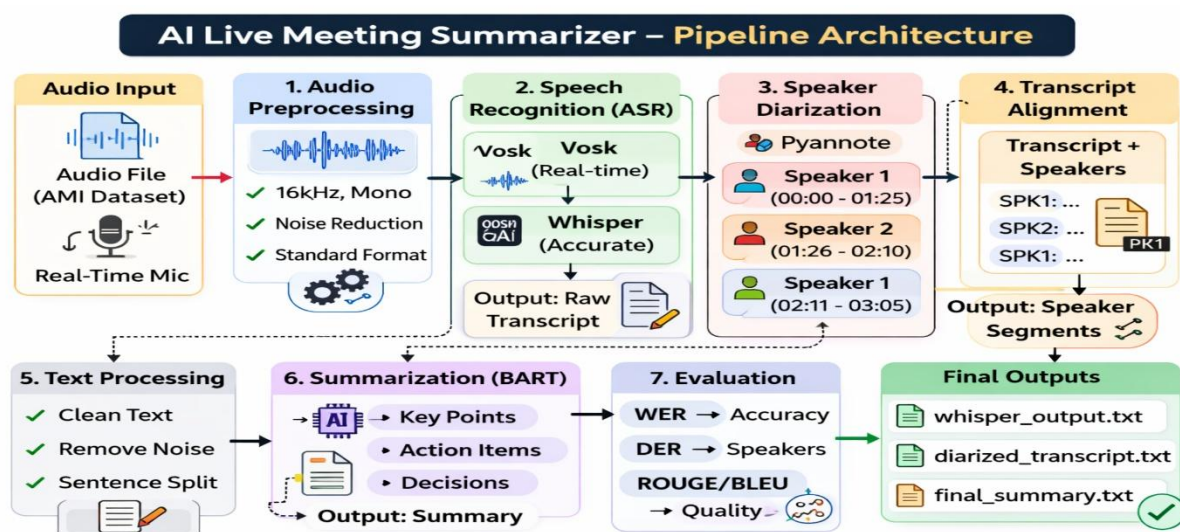
## Integrated Execution

The pipeline is executed using `run_pipeline.py`, which:

- Controls the workflow
- Connects all modules
- Stores outputs

## Final Outputs

- `whisper_output.txt` → Transcript
- `diarized_transcript.txt` → Speaker-wise text
- `final_summary.txt` → Summary



## Challenges

- Synchronizing timestamps
- Managing intermediate data
- Maintaining pipeline speed

## 18. Key Outcome

The project successfully implemented:

- ASR pipeline
- Model comparison
- Real-time transcription
- Speaker diarization
- Summarization system

## 19. Future Work

- Use Whisper Large model
- Apply noise reduction
- Improve diarization
- Enhance summarization

## 20. Conclusion

The AI Live Meeting Summarizer successfully demonstrates a complete pipeline for processing meeting audio. Whisper outperformed Vosk, real-time transcription achieved perfect results in clean environments, and diarization achieved moderate accuracy.

The system is functional and can be improved further for real-world deployment.

**GitHub URL** = <https://github.com/SpringboardMentor050/Live-Meeting-Summarizer-Application/tree/Shivam-Kumar>